Operating Systems

Deadlock

Alok Kumar Jagadev

The Deadlock Problem

- Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process in the set.
- This results in an indefinite waiting state, preventing any progress.

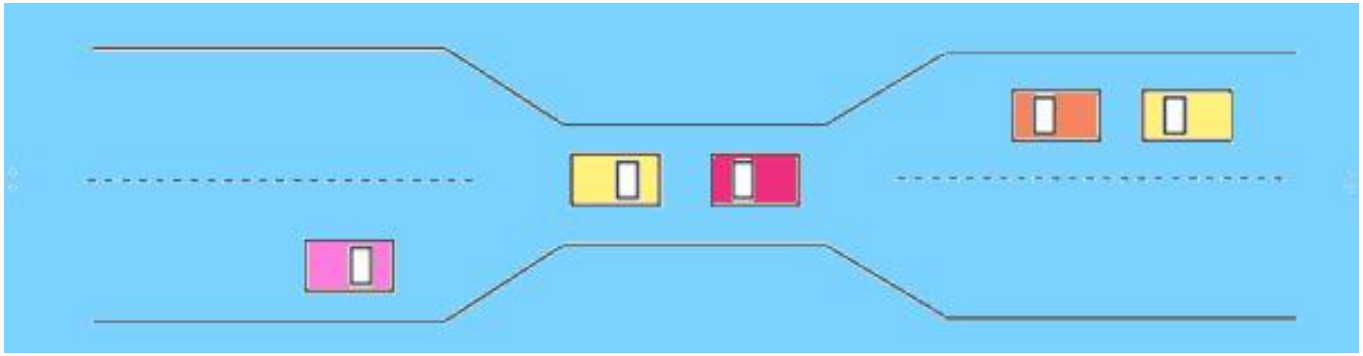
The Deadlock Problem

- Example: Imagine two processes, P1 and P2, and two resources, R1 and R2. Processes P_1 and P_2 each require both resources to complete execution.
  - P1 holds R1 and requests R2.
  - P2 holds R2 and requests R1.
- Since each process is waiting for a resource held by the other, neither can proceed, leading to deadlock.
- Example:
  - semaphores *A* and *B*, initialized to 1 P_0 P_1

    wait (A); wait(B)

    wait (B); wait(A)

Bridge Crossing Example



- Imagine a narrow one-lane bridge where vehicles can only pass in one direction at a time.
- If two vehicles (one from each direction) enter the bridge at the same time, they both get stuck in the middle, waiting for the other to back off.
- Since neither driver wants to reverse, they remain stuck indefinitely, creating a deadlock.

System Model

System Components:

- A system consists of:
  - Processes (P1, P2, P3, ...) – Programs executing concurrently.
  - Resource Types (R1, R2, R3, ...) – CPU, memory, files, I/O devices, etc.
- Each resource type R_i has W_i instances.
- Each process follows a resource allocation sequence:
  - Request: Process requests a resource.
  - Use: Process uses the resource.
  - Release: Process releases the resource after use.
- If resources are not allocated carefully, circular waiting can occur, leading to a deadlock.

Livelock

- Livelock is a situation similar to deadlock, where two or more processes are unable to proceed because they continuously change their state in response to each other, but no useful progress is made.
- Unlike deadlock (where processes remain stuck indefinitely), in livelock, processes keep moving but never complete their task.
- Example: Imagine two people walking towards each other in a narrow hallway:
  - Person A moves left to let Person B pass.
  - Person B also moves left at the same time.
  - Seeing the conflict, both move right at the same time.
  - This cycle keeps repeating, and neither person can cross.
- They keep moving, but neither makes progress.

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

Deadlock Characterization

- Mutual exclusion:
  - A resource can only be used by one process at a time.
  - If another process requests the resource, it must wait until it is released.
  - Example: A printer is being used by one process; other processes must wait.

- Hold and wait:
  - A process holding at least one resource is waiting for additional resources held by other processes.
  - Example:
    - P1 holds R1 and requests R2.
    - P2 holds R2 and requests R1.
    - Both are waiting for each other, leading to a deadlock.

Deadlock Characterization

- No preemption:
  - A resource cannot be forcibly taken away from a process.
  - A process must release the resource voluntarily after completing its task.
  - Example: If a process is using a file, it cannot be forcibly taken away by another process. The second process must wait.
- Circular wait:
  - A set of processes {P1, P2, ..., Pn} exists such that:
  - P1 is waiting for a resource held by P2
  - P2 is waiting for a resource held by P3
  - ...
  - Pn is waiting for a resource held by P1
  - Example (Circular Dependency):
  - P1 → R1 → P2 → R2 → P3 → R3 → P1 (Deadlock)
  - Since all processes are waiting in a cycle, none can proceed.

Difference between Deadlock and Starvation

Sl.	Deadlock	Starvation
1	All processes keep waiting for each other to complete and none get executed	High priority processes keep executing and low priority processes are blocked
2	infinite waiting	long waiting but not infinite
3	Every Deadlock is always a starvation	Every starvation need not be deadlock
4	Resources are blocked by the processes	Resources are continuously utilized by high priority processes
5	Happens when Mutual exclusion, hold and wait, No preemption and circular wait occurs simultaneously	Occurs due to the uncontrolled priority and resource management
6	Also known as Circular wait	Also know as lived lock
7	Can be prevented by avoiding the necessary conditions for deadlock	Can be prevented by Aging

Resource-Allocation Graph

- Resource Allocation Graph (RAG):
  - A graphical representation of the processes and resources in a system.
  - It helps visualize how resources are allocated to processes and whether a deadlock exists.

- Mathematically, a graph is represented as G=(V, E).
  - where V is the set of virtices, E is the set of edges.
- The graph contains nodes (vertices) and edges.

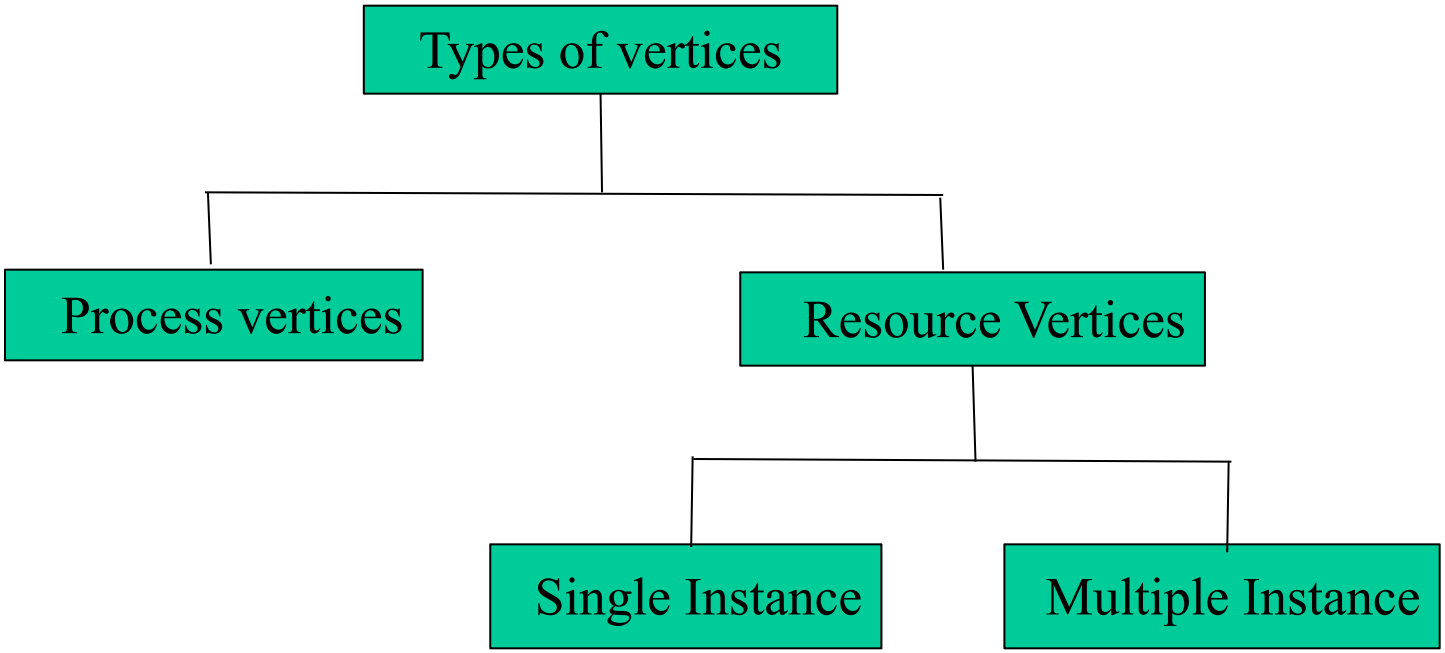
Resource-Allocation Graph

A set of vertices V and a set of edges E.

- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system

- E represented
  - request edge – directed edge $P_i \rightarrow R_j$: An arrow from the process to resource indicates the process is requesting the resource
  - assignment edge – directed edge $R_j \rightarrow P_i$: An arrow from resource to process shows an instance of the resource has been allocated to the process

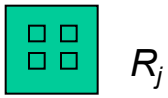
Resource-Allocation Graph



Resource-Allocation Graph

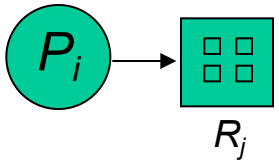
- Process Vertices
  - Process vertices represent the processes
  - Drawn as a circle by mentioning the name of process inside the circle
- Resource Vertices
  - Resource vertices represent the resources
  - may be single instance or multiple instance
  - Drawn as a rectangle by mentioning the dots inside
  - The number of dots inside the rectangle indicates the number of instances of that resource existing in the system
- Process:

- Resource Type with 4 instances:

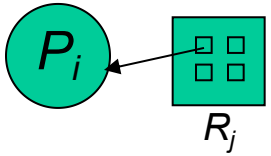


Resource-Allocation Graph

- P_i requests instance of R_j

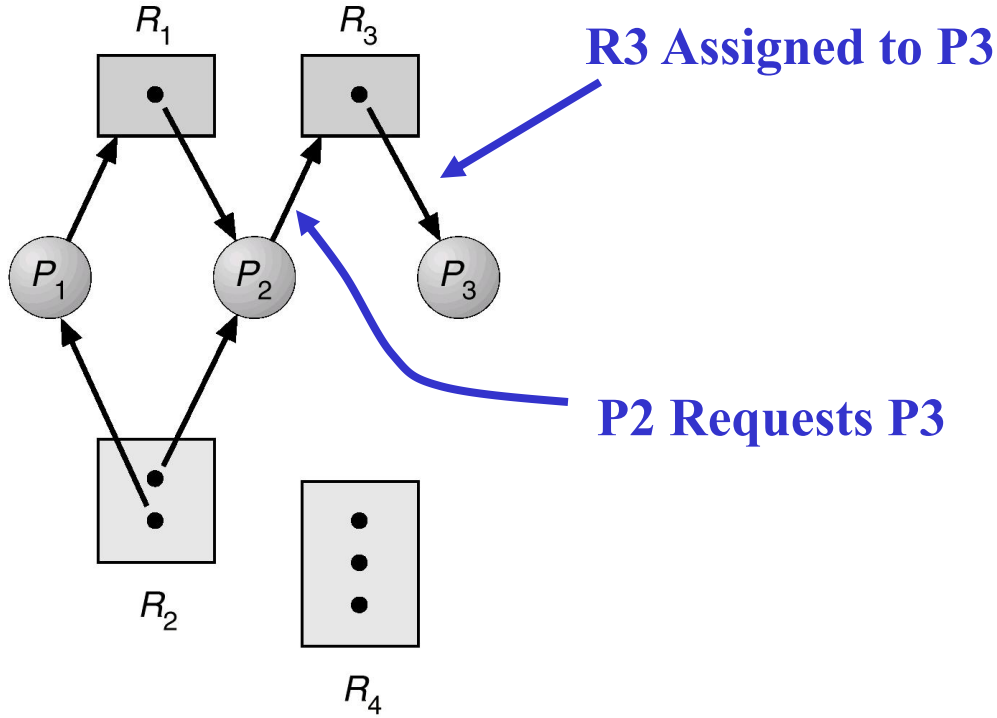


- P_i is holding an instance of R_j



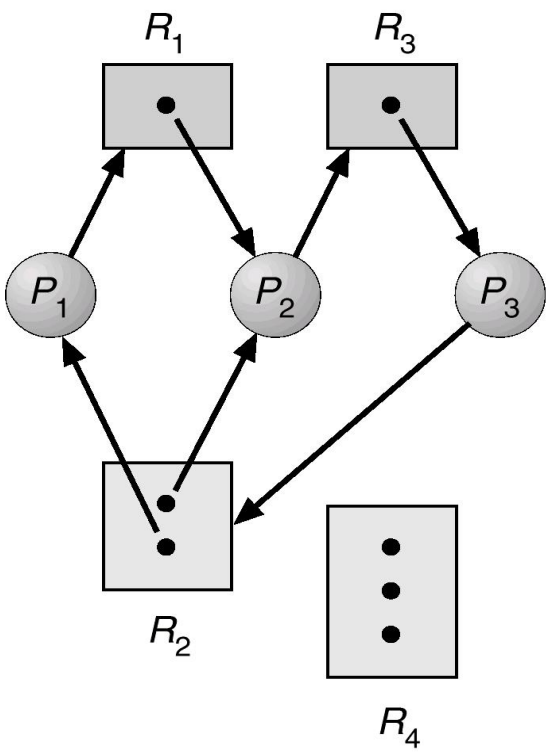
Example of a Resource Allocation Graph

Resource allocation graph

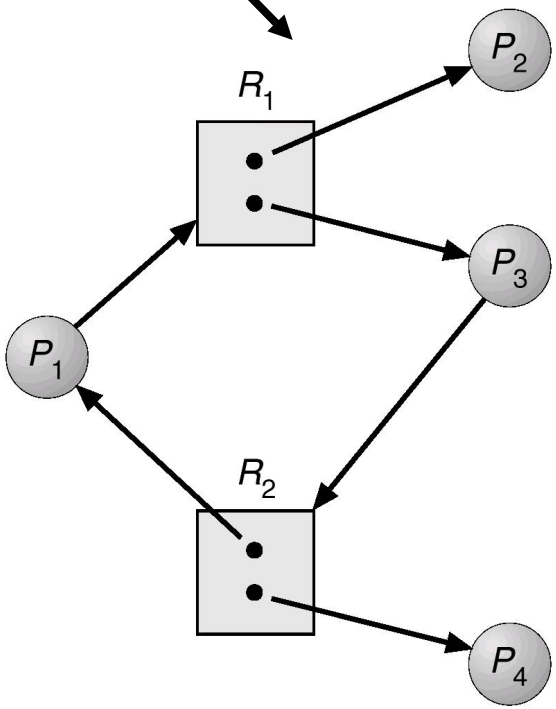


Resource Allocation Graph

Resource allocation graph with a deadlock.



Resource allocation graph with a cycle but no deadlock.



Basic Facts

- If graph contains no cycles ⇒ no deadlock
- If graph contains a cycle ⇒
  - if only one instance per resource type, then deadlock
  - if several instances per resource type, possibility of deadlock

How To Handle Deadlocks – General Strategies

There are three methods:

- Ignore Deadlocks: If deadlock is very rare, then let it happen and reboot the system.
- Ensure deadlock **never** occurs using either

  Prevention Prevent any one of the 4 conditions from happening

  Avoidance Futuristic in approach, it allows all deadlock conditions but predicts impending cycles and prevents risky operations.
- Allow deadlock to happen. This requires using both:

  Detection Know a deadlock has occurred

  Recovery Perform preemption to address the deadlock once it occurs and recover the resources.

Deadlock Ignorance

- Most widely used approach
- Being used by many operating systems mainly for end users
- In this approach, the OS assumes that deadlock never occurs
- It simply ignores deadlock
- Best suitable for a single end user system
  - where User uses the system only for browsing and all other normal stuff

Deadlock Ignorance

- Always a tradeoff between Correctness and performance
- OS like Windows and Linux mainly focus upon performance
- Performance of the system decreases if it uses deadlock handling mechanism all the time
  - if deadlock happens 1 out of 100 times then it is completely unnecessary to use the deadlock handling mechanism all the time
- In these types of systems,
  - the user has to simply restart the computer in the case of deadlock
- Windows and Linux are mainly using this approach

Deadlock Prevention

- Deadlock Prevention is a strategy that aims to prevent deadlock from ever occurring.
- Deadlock happens only when Mutual Exclusion, hold and wait, No preemption and circular wait hold simultaneously
- If it is possible to violate one of the four conditions at any time then the deadlock can never occur in the system

Deadlock Prevention

Mutual Exclusion

- Condition: A resource can only be held by one process at a time.
- Prevention Strategy: In some cases, the system can ensure resources are shared or not mutually exclusive (e.g., read-only access to a resource can be given to multiple processes, i.e. mutual exclusion property can be violeted).
- Example:
  - In some database systems, multiple processes can read data simultaneously, but only one can write to it at a time.
  - By ensuring that only certain resources are locked (e.g., for writing), deadlock is prevented.

Deadlock Prevention

Hold and Wait

- Condition: A process holds at least one resource and is waiting to acquire additional resources held by other processes.
- Prevention Strategy: Avoid hold and wait: Find some mechanism by which a process
  - either doesn't hold any resource
  - or doesn't wait
- Example:
  - A process that needs resources R1 and R2 must request both at the same time.
  - If it cannot acquire both, it cannot hold R1 while waiting for R2.
  - This prevents the situation where a process holds one resource and waits for another.

Deadlock Prevention

- Two methods
  - Method-1: A process must be assigned all the necessary resources before the execution starts
  - Method-2: Allow process to request resources only when the process has none

Deadlock Prevention

Hold and Wait

Example: Consider a process that copies data from DVD drive to a file on disk and then prints to a printer

- Method1: request DVD drive, Disk, and Printer before it execute
  - It will hold the Printer for entire execution, even though it needs the printer only at the end
- Method2: request only for DVD drive and Disk initially. It finishes the copy step and release both resource. The process must then again request for disk and printer to finish the job.

- Low resource utilization
- starvation possible, (if a process needs several popular resources)

Deadlock Prevention

No Preemption

- Condition: A resource cannot be preempted (taken away from a process forcibly).
- Prevention Strategy: Allow preemption, meaning that if a process is holding resources and is waiting for others, the system can take away its current resources and give them to other processes.
- Example: If process P1 holds resource R1 and requests R2, but process P2 holds R2 and requests R1, the system can preempt R1 from P1 and allocate it to P2. P1 will then have to restart once it gets R2, but deadlock is prevented.

Deadlock Prevention

No Preemption

- Not a good approach at all
  - Take a resource away which is being used by the process then all the work which it has done till now can become inconsistent
- Example: Consider a printer is being used by any process.
  - If the printer is taken away from that process and assign it to some other process then all the data which has been printed can become inconsistent and ineffective and
  - also the fact that the process cannot start printing again from where it has left which causes performance inefficiency

Deadlock Prevention

Circular Wait

- Condition: A set of processes {P1, P2, ..., Pn} exists such that each process is waiting for a resource held by the next process in the chain, forming a cycle.
- Prevention Strategy: Eliminate circular wait by defining a global ordering of resources. Processes must request resources in this predefined order, preventing cycles.

Deadlock Prevention

Circular Wait

– First, create a global ordering of each resource, for example:
  F(tape drive) = 1, F(disk drive) = 5, F(Printer) =12

Then two rules for Processes to request Resources need to be followed:

i. Each process can request resources only in an increasing order

ii. Whenever a process requests a resource Rj, it must have already released any resources Ri such that $F(R_i) > F(R_j)$

- Let the set of processes involved in the circular wait condition be $\{P_0, P_1, .., P_n\}$, where P_i is waiting for a resource R_i, which is held by P_{i+1} (P_n is waiting for R_n which held by P_0)
- Process P_{i+1} by holding R_i requesting resource R_{i+1} while $F(R_i) < F(R_{i+1})$ for all i.
- This implies $F(R_0)<F(R_1)<\ldots<F(R_n)<F(R_0)$
- By transitivity $F(R_0) < F(R_0)$ which is impossible

Deadlock Prevention

Condition	Approach	Is Practically Possible?
Mutual Exclusion	Spooling	×
Hold and Wait	Request for all resources initially	×
No Preemption	Take away resources from processes	×
Circular Wait	Assign a priority to all resources and order resources numerically	√

Among all the methods, violating Circular wait is the only approach that can be implemented practically

Deadlock Avoidance

- Possible side effects of preventing deadlock are
  - low device utilization and
  - reduce system throughput
- Deadlock avoidance is a technique used to dynamically determine whether granting a resource request will lead to deadlock.
- deadlock avoidance allows resource requests carefully after analyzing.
- This approach requires knowledge of the future resource requests of processes
  - makes it more flexible but also more complex than deadlock prevention.

Deadlock Avoidance

- System requires additional a priori information :
  - Each process declares the maximum number of resources of each type that it may require
  - deadlock avoidance algorithm continuously evaluates the resource allocation state to prevent the possibility of a circular wait.
  - Resource-allocation state is defined by
    - number of available and allocated resources, and
    - maximum demands of the processes

Safe State

- When a process requests an available resource, the system must determine whether granting the request immediately will keep the system in a safe state.
- A system is in a safe state if it can allocate resources to every process in some order without getting into a deadlock.
- If no such sequence exists, then the system state is said to be unsafe

Safe State (Contd…)

- For example, consider a system with 12 magnetic tape drives and three processes:
  - P0 requires 10 tapes, P1 requires 4, and P2 requires 9.
  - Currently, P0 holds 5 tapes, P1 holds 2, and P2 holds 2.

-

	Max needs	Currently Holding	Need
P_0	10	5	5
P_1	4	2	2
P_2	9	2	7

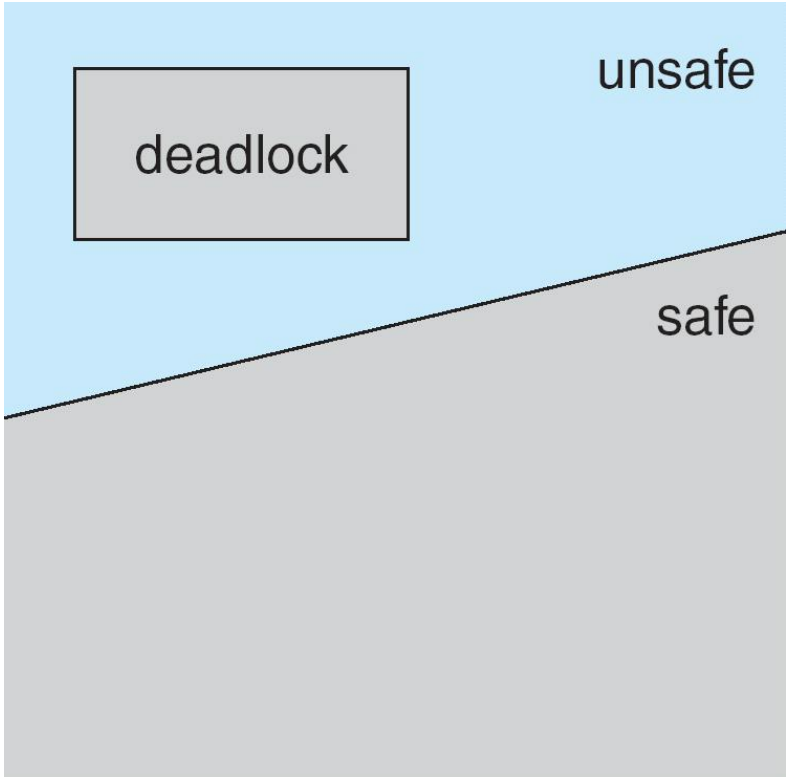
- The system is in safe state, since $<P_1, P_0, P_2>$ satisfied safe state condition

What if P_2 currently holding one more tape and has that one?

Basic Facts

- If a system is in safe state ⇒ no deadlocks
- If a system is in unsafe state ⇒ possibility of deadlock
- Avoidance ⇒ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State



Deadlock Avoidance

- The algorithm ensures that the system always remains in a safe state.
- A process might need to wait for resource allocation even when a resource is available.
- This can lead to lower resource utilization.

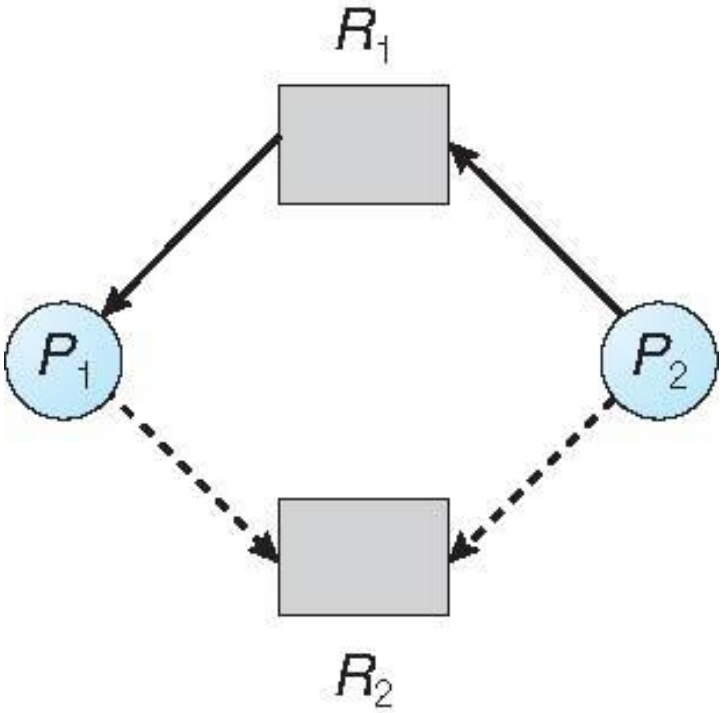
Avoidance algorithms

- Single instance of a resource type
  - Use a resource-allocation graph
- Multiple instances of a resource type
  - Use the banker's algorithm

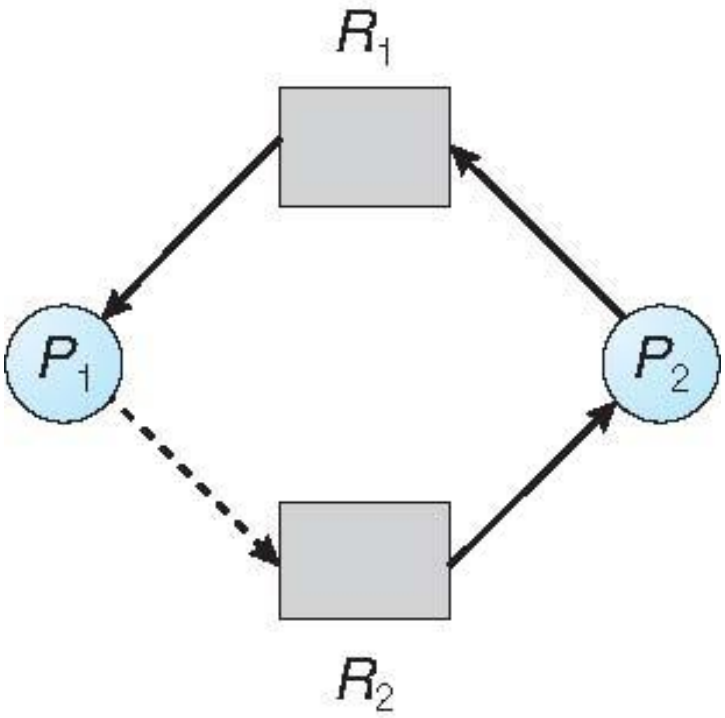
Resource-Allocation Graph Scheme

- Claim edge: $P_i \rightarrow R_j$ indicated that process P_i may request resource R_j; represented by a dashed line
  - Claim edge converts to request edge when a process requests a resource
  - Request edge converted to an assignment edge when the resource is allocated to the process
  - When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system

Resource-Allocation Graph



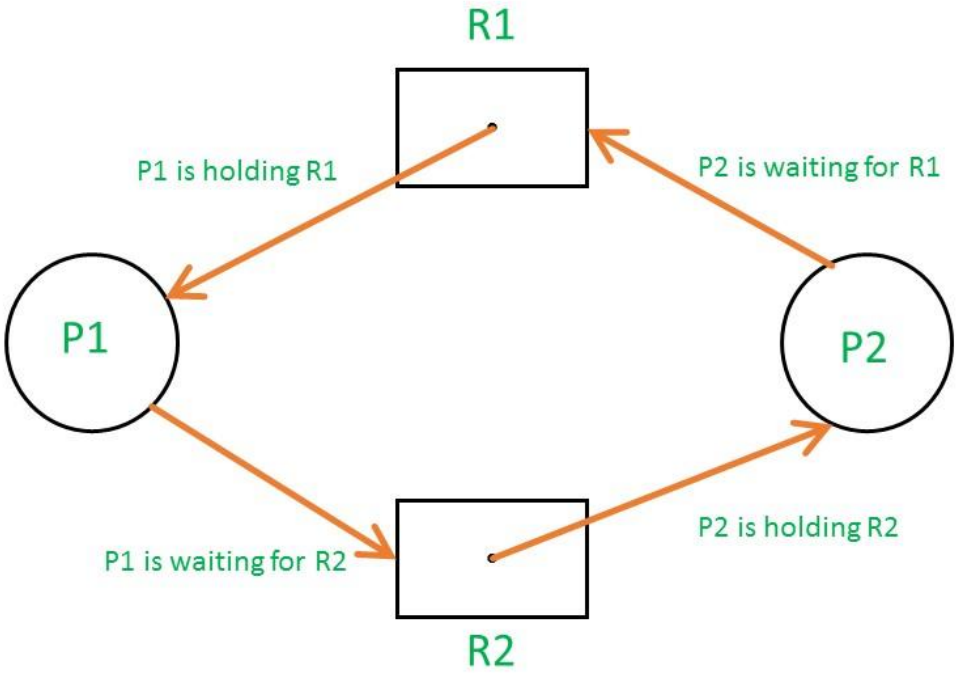
Unsafe State In Resource-Allocation Graph



Resource-Allocation Graph Algorithm

- Suppose process P_i requests a resource R_j
- A request is granted only if converting the request edge into an assignment edge does not create a cycle in the resource allocation graph.

Example: RAG Algorithm

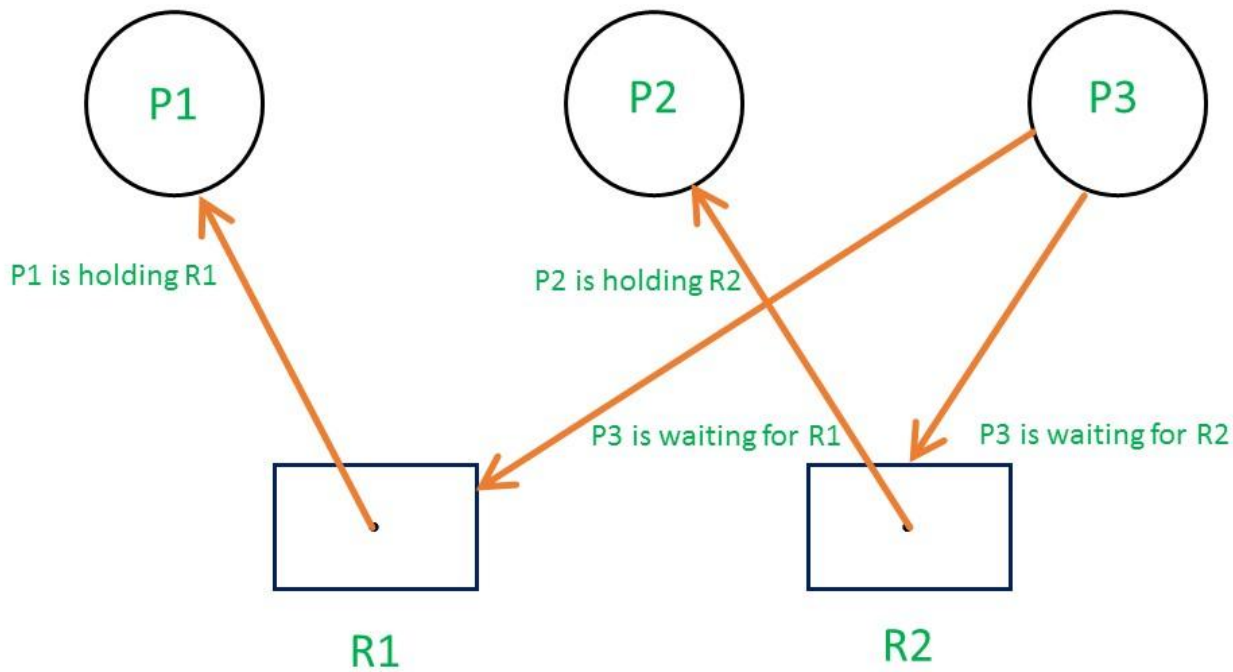


SINGLE INSTANCE RESOURCE TYPE WITH DEADLOCK

Example: RAG Algorithm

- If there is a cycle in the RAG and each resource in the cycle are only one instance, then the processes will be in deadlock.
- For example,
  - if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.

Example: RAG Algorithm

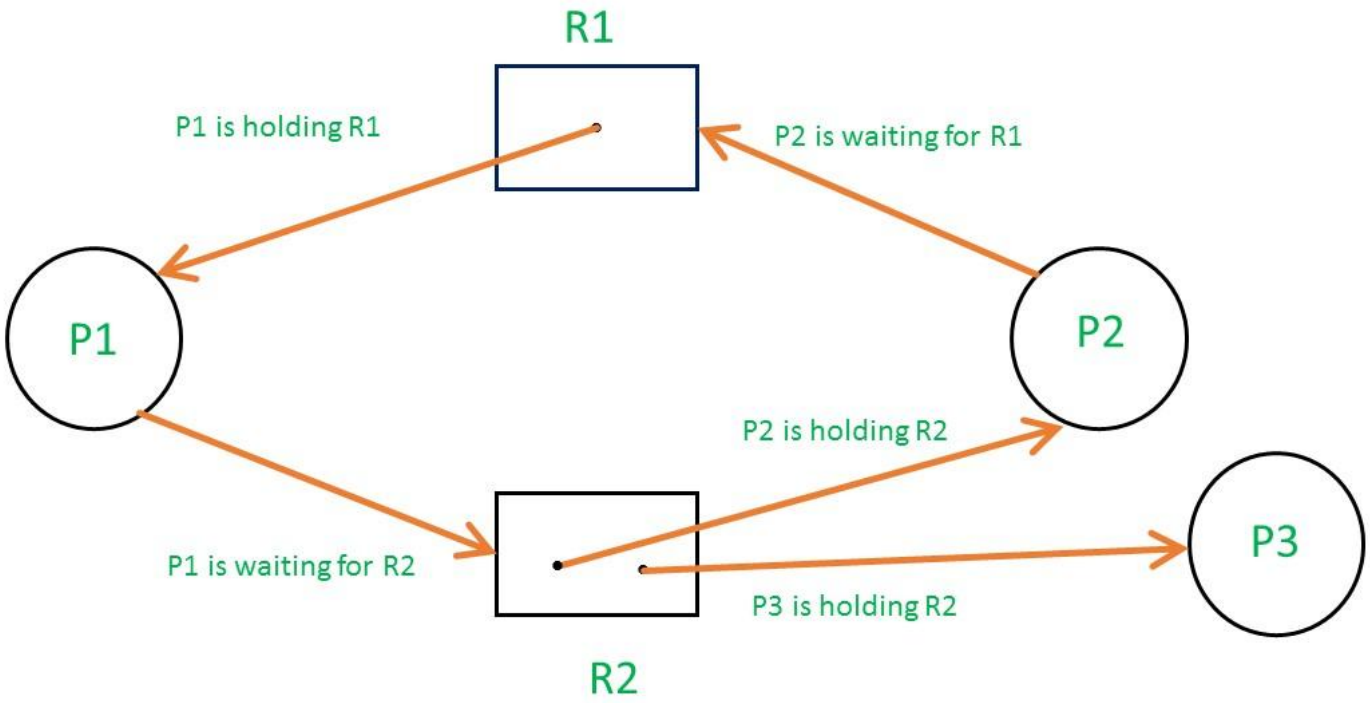


SINGLE INSTANCE RESOURCE TYPE WITHOUT DEADLOCK

Example: RAG Algorithm

- Processes P1 and P2 acquiring resources R1 and R2 while process P3 is waiting to acquire both resources.
- In this example, there is no deadlock because there is no circular dependency.
- So, cycle in single-instance resource type is the sufficient condition for deadlock.

Example: RAG Algorithm



MULTI INSTANCES WITHOUT DEADLOCK

Example: RAG Algorithm

- it is not possible to say the RAG is in a safe state or in an unsafe state.
- So to see the state of this RAG, construct the allocation matrix and request matrix.

Example: RAG Algorithm

Process	Allocation		Request	
	Resource		Resource	
	R1	R2	R1	R2
P1	1	0	0	1
P2	0	1	1	0
P3	0	1	0	0

Example: RAG Algorithm

- The number of processes: P1, P2 & P3 and the number of resource types: R1 & R2.

Allocation matrix:

- For constructing the allocation matrix, just go to the resources and see to which process it is allocated.
- R1 is allocated to P1, therefore write 1 in allocation matrix and similarly, R2 is allocated to P2 as well as P3 and for the remaining element just write 0.

Request matrix:

- In order to find out the request matrix, you have to go to the process and see the outgoing edges.
- P1 is requesting resource R2, so write 1 in the matrix and similarly, P2 requesting R1 and for the remaining element write 0.
- So now available resource is = (0, 0).

Banker's Algorithm

- The Banker's algorithm is named for its application in banking systems, where it ensures that the bank never allocates its available cash in a way that prevents it from meeting the needs of all its customers.

- Resources have multiple instances.
- Each process must declare its maximum resource requirement in advance.
- A process may need to wait when requesting a resource.
- Once a process acquires all required resources, it must release them within a finite time.

Banker's Algorithm

Two algorithms need to be discussed:

1. Safety state check algorithm

2. Resource request algorithm

Data Structures for the Banker's Algorithm

- Let n = number of processes, and m = number of resources types.
- Available: Vector of length m. If available $[j] = k$, there are k instances of resource type R_j available
- Max: $n \times m$ matrix. If $Max\ [i, j] = k$, then process P_i may request at most k instances of resource type R_j
- Allocation: $n \times m$ matrix. If Allocation$[i, j] = k$ then P_i is currently allocated k instances of R_j
- Need: $n \times m$ matrix. If $Need[i, j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need\ [i, j] = Max[i, j] – Allocation\ [i, j]$$

Safety Algorithm

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively. Initialize:

   $Work = Available$

   $Finish\ [i] = false$ for $i = 0, 1, \ldots, n-1$

2. Find an *i* such that both:

   (a) $Finish\ [i] = false$

   (b) $Need_i \leq Work$

   If no such *i* exists, go to step 4

3. $Work = Work + Allocation_i$

   $Finish[i] = true$

   go to step 2

4. If $Finish\ [i] == true$ for all *i*, then the system is in a safe state

Resource-Request Algorithm for Process P_i

Request = request vector for process P_i. If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2.

   Otherwise, raise error condition, since process has exceeded its maximum claim

2. If $Request_i \leq Available$, go to step 3.

   Otherwise P_i must wait, since resources are not available

3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request\ ;$$
$$Allocation_i = Allocation_i + Request_i\ ;$$
$$Need_i = Need_i - Request_i\ ;$$

- *If safe $\Rightarrow$ the resources are allocated to P_i*
- *If unsafe $\Rightarrow$ P_i must wait, and the old resource-allocation state is restored*

Example: Banker's Algorithm

- Considering a system with five processes P0 through P4 and three resource types A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example: Banker's Algorithm

- What will be the content of the Need matrix?
- Need [i, j] = Max [i, j] – Allocation [i, j]
- So, the content of Need Matrix is:

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Example: Banker's Algorithm

- Is the system in a safe state? If Yes, then what is the safe sequence?
- Applying the Safety algorithm on the given system,

m=3, n=5 — Step 1 of Safety Algo

Work = Available

Work = | 3 | 3 | 2 |

	0	1	2	3	4
Finish =	false	false	false	false	false

For i = 0 — Step 2

$Need_0$ = 7, 4, 3 (7,4,3 > 3,3,2)

Finish [0] is false and $Need_0$ > Work

So P_0 must wait — But Need ≤ Work

For i = 1 — Step 2

$Need_1$ = 1, 2, 2 (1,2,2 < 3,3,2)

Finish [1] is false and $Need_1$ < Work

So P_1 must be kept in safe sequence

Step 3

Work = Work + $Allocation_1$ (3, 3, 2 + 2, 0, 0)

	A	B	C
Work =	5	3	2

	0	1	2	3	4
Finish =	false	true	false	false	false

For i = 2 — Step 2

$Need_2$ = 6 , 0, 0 (6, 0, 0 > 5,3, 2)

Finish [2] is false and $Need_2$ > Work

So P_2 must wait

For i=3 — Step 2

$Need_3$ = 0, 1, 1

Finish [3] = false and $Need_3$ < Work (0, 1, 1 < 5, 3, 2)

So P_3 must be kept in safe sequence

Step 3

Work = Work + $Allocation_3$ (5, 3, 2 + 2, 1, 1)

	A	B	C
Work =	7	4	3

	0	1	2	3	4
Finish =	false	true	false	true	false

For i = 4 — Step 2

$Need_4$ = 4, 3, 1

Finish [4] = false and $Need_4$ < Work (4, 3, 1 < 7, 4, 3)

So P_4 must be kept in safe sequence

Step 3

Work = Work + $Allocation_4$ (7, 4, 3 + 0, 0, 2)

	A	B	C
Work =	7	4	5

	0	1	2	3	4
Finish =	false	true	false	true	true

For i = 0 — Step 2

$Need_0$ = 7, 4, 3

Finish [0] is false and Need < Work (7, 4, 3 < 7, 4, 5)

So P_0 must be kept in safe sequence

Step 3

Work = Work + $Allocation_0$ (7, 4, 5 + 0, 1 , 0)

	A	B	C
Work =	7	5	5

	0	1	2	3	4
Finish =	true	true	false	true	true

For i = 2 — Step 2

$Need_2$ = 6 , 0, 0

Finish [2] is false and $Need_2$ < Work (6, 0, 0 < 7, 5, 5)

So P_2 must be kept in safe sequence

Step 3

Work = Work + $Allocation_2$ (7, 5, 5 + 3, 0, 2)

	A	B	C
Work =	10	5	7

	0	1	2	3	4
Finish =	true	true	true	true	true

Step 4

Finish [i] = true for 0 ≤ i ≤ n

Hence the system is in Safe state

The safe sequence is P_1, P_3 , P_4 , P_0, P_2

Example: P_1 Request (1,0,2)

- What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?

A B C
$Request_1$ = 1, 0, 2

To decide whether the request is granted we use Resource Request algorithm

Step 1

1, 0, 2 1, 2, 2
$Request_1 < Need_1$

Step 2

1, 0, 2 3, 3, 2
$Request_1 < Available$

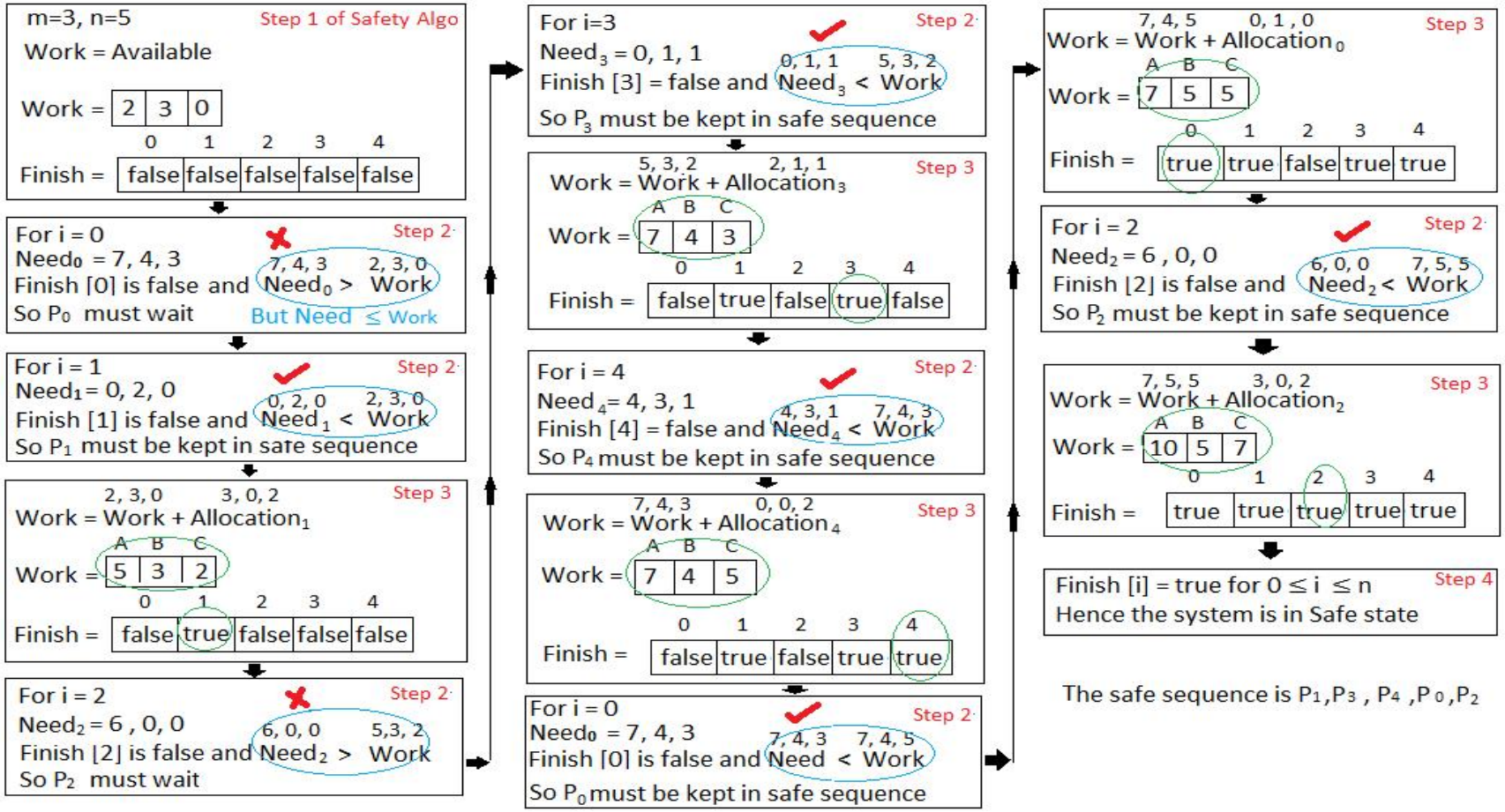
Step 3

$Available = Available - Request_1$
$Allocation_1 = Allocation_1 + Request_1$
$Need_1 = Need_1 - Request_1$

Process	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Example: P_1 Request (1,0,2)

- We must determine whether this new system state is safe. To do so, we again execute Safety algorithm on the above data structures.



Problem

Process	Max				Allocation				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	6	0	1	2	4	0	0	1	3	2	1	1
P1	2	7	5	0	1	1	0	0				
P2	2	3	5	6	1	2	5	4				
P3	1	6	5	3	0	6	3	3				
P4	1	6	5	6	0	2	1	2				

Using Banker's algorithm, answer the following questions:

i) How many resource types A, B, C, D are there?

ii) What are the contents of need matrix?

iii) Find if the system is in safe state? If it is, find the safe sequence.

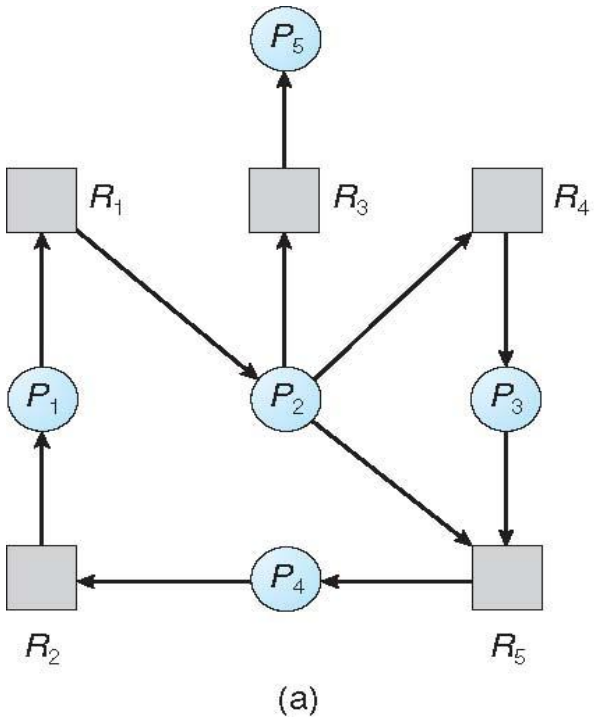
Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

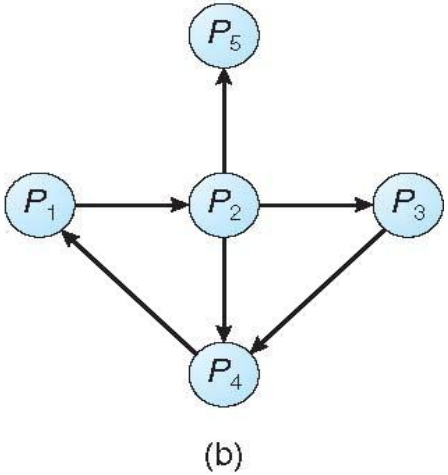
Single Instance of Each Resource Type

- Maintain *wait-for* graph
  - Nodes are processes
  - $P_i \rightarrow P_j$ if P_i is waiting for P_j
- Periodically invoke an algorithm that searches for a cycle in the graph.
  - If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph

Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph



Corresponding wait-for graph

Several Instances of a Resource Type

- Available: A vector of length m indicates the number of available resources of each type.
- Allocation: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- Request: An $n \times m$ matrix indicates the current request of each process. If *Request* $[i][j] = k$, then process P_i is requesting k more instances of resource type.R_j.

Detection Algorithm

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively Initialize:

   (a) *Work* = *Available*

   (b) For $i = 1, 2, \ldots, n$, if $Allocation_i \neq 0$, then *Finish*[i] = false; otherwise, *Finish*[i] = *true*

2. Find an index *i* such that both:

   (a) *Finish*[*i*] == *false*

   (b) $Request_i \leq Work$

   If no such *i* exists, go to step 4

Detection Algorithm (Cont.)

3. $Work = Work + Allocation_i$
   $Finish[i] = true$
   go to step 2

4. If $Finish[i]$ == false, for some i, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i]$ == *false*, then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state

Example of Detection Algorithm

- Five processes P_0 through P_4; three resource types A (7 instances), *B* (2 instances), and *C* (6 instances)
- Snapshot at time T_0:

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence <P_0, P_2, P_3, P_4, P_1> will result in *Finish*[*i*] = true for all *i*

Example (Cont.)

- P_2 requests an additional instance of type C

	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

- State of system?
  - Can reclaim resources held by process P_0, but insufficient resources to fulfill other processes; requests
  - Deadlock exists, consisting of processes P_1, P_2, P_3, and P_4

Detection-Algorithm Usage

- When, and how often, to invoke depends on:
  - How often a deadlock is likely to occur?
  - How many processes will need to be rolled back?
    - one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes "caused" the deadlock.

Recovery from Deadlock

- Process Termination
- Resource Preemption

Recovery from Deadlock

Process Termination

- Could delete all the processes in the deadlock -- this is expensive.
- Delete one at a time until deadlock is broken (time consuming).
- Select who to terminate based on priority, time executed, time to completion, needs for completion, or depth of rollback
- In general, it's easier to preempt the resource, than to terminate the process.

Recovery from Deadlock

Resource Preemption

- Selecting a victim – minimize cost
- Rollback – return to some safe state, restart process for that state
- Problem: starvation –Prevent one process from always being the one preempted

Example of Deadlock Detection and Recovery

- Scenario: Consider three processes P1, P2, P3 and two resources R1, R2. The allocation is as follows:

Process	Allocated (R1, R2)	Requested (R1, R2)
P1	(1, 0)	(0, 1)
P2	(0, 1)	(1, 0)
P3	(1, 1)	(0, 0)

- Available resources: (0,0)

- Wait-for Graph contains a cycle: P1→P2→P1.
- Deadlock detected because each process is waiting for a resource held by another.

- Recovery Strategy:
- Process Termination: Abort P1 or P2 to break the cycle.
- Resource Preemption: Take R1 from P3 and assign it to P2, allowing progress.